

Lezione 14

14.1 ALGORITMO DI DISKSTRA

DISKSTRA(G, s, w)

```
1. INITIALIZE-SINGLE-SOURCE( $G, s$ )
2.  $S := \emptyset$ 
3.  $Q := \emptyset$ 
4. for each vertex  $u \in V$ 
5.   INSERT( $Q, u$ )
6. while  $Q \neq \emptyset$ 
7.    $u = \text{EXTRACT-MIN}(Q)$ 
8.    $S := S \cup \{u\}$ 
9.   for each  $v \in \text{Adj}[u]$ 
10.    RELAX( $u, v, w$ )
11.    if the call of RELAX decreased  $v.d$ 
12.      DECREASE-KEY( $Q, v, v.d$ )
```

$O(V)$

$O(1)$ } $O(V)$

dipende dal tipo
di code, $O(V)$

$O(1)$ } $O(\text{Adj}(u))$ } $O(V)$

Oss: L'algoritmo mantiene l'invariante $Q = V \setminus S$ dalle linee 6 su per.

THM (CORRETTEZZA DELL'ALGORITMO DI DISKSTRA): L'algoritmo di Dijkstra

eseguito su un grafo orientato $G=(V, E)$ con una funzione di peso $w: E \rightarrow \mathbb{R}^+$
a valori nonnegativi e vertice sorgente s , termina con $u.d = \delta(s, u)$
 $\forall u \in V$.

Proof: Dimostriamo che all'inizio di ogni iterazione del ciclo while (linee 6-12) $v.d = \delta(s, v) \quad \forall v \in S$. Questo sarà sufficiente a provare

la correttezza dell'algoritmo perché quando l'algoritmo termina $S=V$,
così che $v.d = \delta(s, v) \quad \forall v \in V$.

Questo fatto verrà dimostrato per induzione sul numero di iterazioni

del ciclo whose gré avvenute, tale numero è uguale a $|S|$ all'inizio di ogni iterazione.

$$\boxed{v.d = \delta(s, v) \quad \forall v \in S}$$

Paso base (caso 2):

- $|S|=0 \Rightarrow S=\emptyset \Rightarrow$ il claim è vero banalmente.
- $|S|=1 \Rightarrow S=\{s\}$ e $s.d=0=\delta(s, s)$. $\checkmark$

Ipotesi induttiva: $v.d = \delta(s, v) \quad \forall v \in S$

Paso induttivo: ora viene estratto $u \in Q = V \setminus S$ e viene aggiunto ad S , dobbiamo mostrare che $u.d = \delta(s, u)$ a questo istante.

- Se non ci sono cammini da s a u , allora per la no-path property $u.d = \infty = \delta(s, u)$. $\checkmark$

- Se esiste un cammino da s a u , allora sia y il primo vertex su un cammino minimo da s a u che non sia in S e sia x il suo predecessore (potrebbe accadere che $x=s$ o $y=u$). Siccome y appare non successivamente a u e i pesi sono nonnegativi

(1) $\boxed{\delta(s, y) \leq \delta(s, u)}$. Inoltre, EXTRACT-MIN (linea 7) ha estratto u perché $u.d$ è minimo in $Q = V \setminus S$, quindi poiché $y \notin S$

(3) $\boxed{u.d \leq y.d}$ e per la upper-bound property $\boxed{\delta(s, u) \leq u.d}$. (2)

Siccome $x \in S$, per l'ipotesi induttiva $x.d = \delta(s, x)$. Durante l'iterazione che ha aggiunto x ad S (x, y) è stato inserito

quindi $\boxed{y.d = \delta(s, y)}$ per la convergence property.

Allora,

$$\delta(s, y) \stackrel{(1)}{\leq} \delta(s, u) \stackrel{(2)}{\leq} u.d \stackrel{(3)}{\leq} y.d = \delta(s, y) \stackrel{(4)}{\Rightarrow} u.d = \delta(s, u)$$

e per la upper-bound property sappiamo che questo valore non cambia più. $\square$

14.12 Analisi dell'algoritmo di Dijkstra

L'algoritmo di Dijkstra mantiene la coda di priorità Q invocando tre operazioni: `INSERT` (linea 5), `EXTRACT-MIN` (linea 7) e `DECREASE-KEY` (linea 11).

L'algoritmo invoca `INSERT` ed `EXTRACT-MIN` per ogni vertex.

Ogni vertex v è aggiunto ad S esattamente una volta, ogni arco in $\text{Adj}[u]$ è esaminato nel ciclo for (linee 9-12) esattamente una volta durante il corso dell'intero algoritmo.

$\sum_{v \in V} |\text{Adj}[v]| = |E| \Rightarrow$ questo ciclo viene iterato per $|E|$ volte.

Il tempo computazionale dell'algoritmo di Dijkstra dipende dalle specifiche implementazioni delle code di priorità Q .

- Se implementiamo Q come un array che enumera i vertici e memorizza $v.d$ nella v -esima posizione dell'array

$\Rightarrow \text{EXTRACT-MIN} \in O(V)\text{-Time} \Rightarrow$ globalmente il tempo computazionale è $O(|V|^2 + |E|)$
 $= O(V^2)$

- Possiamo implementare Q con un min-heap e ottenere tempo globale $O((V+E) \log V)$, che è $O(E \log V)$ se $|E| = \Omega(V)$.

$\downarrow$ in questo caso DECREASE-KEY impiega $\log V$

COR: Dopo avere eseguito Dijkstra su un grafo orientato e pesato

G con pesi nonnegativi e sorgente s , il sottografo dei predecessori G_π è un albero dei cammini minimo radicato ad s .

Proof: Immediata dalla dimostrazione precedente e dalle predecessori subgraph property. $\square$

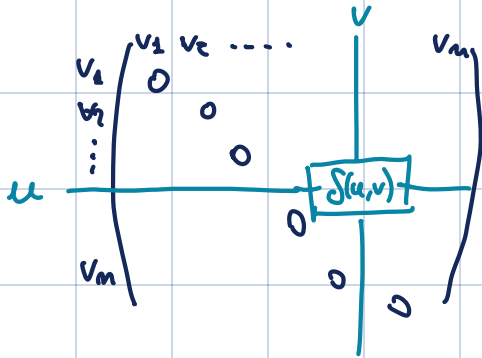
14.2 ALL-PAIRS-SHORTEST-PATH

ALL-PAIRS-SHORTEST-PATH

input: $G=(V,E)$ grafo orientato, $w:E \rightarrow \mathbb{R}$ funzione di peso sugli archi.

goal: trovare per ogni $u,v \in V$ un cammino minimo da u a v .

L'output di un algoritmo che risolve questo problema è tipicamente una tabella (matrice) in cui le righe e le colonne sono indizzate dai vertici di G e l'elemento alla riga u e colonna v è il peso di un cammino minimo da u a v .



Applicazione (famosa): calcolo del diametro di una rete.

DSS: Potremmo risolvere ALL-PAIRS-SHORTEST-PATH eseguendo uno degli algoritmi

per SINGLE-SOURCE-SHORTEST-PATH per $|V|$ volte.

- Se gli archi hanno peso nonnegativo, possiamo usare l'algoritmo di

Dijkstra:

- implementando la coda di priorità con un array lineare otteniamo un algoritmo con runtime $O(V^3 + VE)$ che è $O(V^3)$;
- implementando la coda con un min-heap otteniamo tempo computazionale uguale a $O(V(V+E)\log V)$ che se $|E| = \Omega(V)$ diventa $O(VE\log V)$, inferiore a $O(V^3)$ se il grafo è sparso.

- Se il grafo contiene archi di peso negativo, possiamo usare Bellman-Ford su ogni vertice, ottenendo un tempo computazionale uguale a $O(V^2E)$ che su un grafo denso diventa $O(V^4)$.

Domanda: Possiamo risolvere ALL-PAIRS-SHORTEST-PATH in un modo più efficiente?

Notazione e Rappresentazione

Assumiamo che il grafo preso in input $G=(V,E)$ sia rappresentato dalla sua matrice di adiacenze (per SINGLE-SOURCE-SHORTEST-PATH era stato conveniente usare le liste di adiacenze).

Enumeriamo i vertici di G , $V=\{1,2,\dots,n\}$, $|V|=n$.

Allora l'input è rappresentato dalla matrice $W=(w_{ij})$ di ordine $n \times n$ definita da

$$w_{ij} = \begin{cases} 0 & \text{se } i=j \\ w(i,j) & \text{se } i \neq j \text{ e } (i,j) \in E \\ \infty & \text{se } i \neq j \text{ e } (i,j) \notin E. \end{cases}$$

Il grafo può avere archi di peso negativo, ma assumiamo che esso non contenga cicli di peso negativo.

Una soluzione completa a ALL-PAIRS-SHORTEST-PATH include non solo le tabelle coi pesi dei cammini minimi, ma anche una matrice $\Pi = (\pi_{ij})$ dei predecessori, con $\pi_{ij} = \text{NIL}$ se $i=j$ o se non ci sono cammini da i a j ($\delta(i,j) = \infty$) e π_{ij} è uguale al predecessore di j in un cammino minimo da i , altrimenti.

La i -esima riga di Π induce un albero dei cammini minimi radicato a i .

La funzione PRINT-ALL-PAIRS-SHORTEST-PATH stampa un cammino minimo da i a j .

PRINT-ALL-PAIRS-SHORTEST-PATH (Π, i, j)

1. if $i=j$
2. print i
3. else if $\pi_{ij} = \text{NIL}$
4. print "non esistono cammini da i a j "
5. else PRINT-ALL-PAIRS-SHORTEST-PATH (Π, i, π_{ij})
6. print j

Oss: I sottocammini di un cammino minimo sono minimi, ovvero tutti i problemi di cammino minimo hanno la proprietà di sottostruttura ottima (e non esistono cicli di peso negativo).

14.2.1. APPROCCIO BASATO SULLA PROGRAMMAZIONE DINAMICA E LA MOLTIPLICAZIONE DI MATRICI

Di serve una formulazione ricorsiva del problema.

Sia $l_{ij}^{(z)}$ il peso di un cammino minimo da i a j che contiene al più z archi.

$$l_{ij}^{(0)} = \begin{cases} 0 & i=j \\ \infty & i \neq j \end{cases}$$

Per $z \geq 1$, un modo per ottenere un cammino minimo da i a j con al più z archi è considerare un cammino minimo con al più $z-1$ archi, così da $l_{ij}^{(z)} = l_{ij}^{(z-1)}$; oppure, considerare un cammino minimo con al più $z-1$ archi da i a un vertice k , per qualche $1 \leq k \leq n$, e poi aggiungere l'arco (k, j) , così $l_{ij}^{(z)} = l_{ik}^{(z-1)} + w_{kj}$.

Allora, possiamo completare la nostra formulazione ricorsiva

$$z \geq 1, \quad l_{ij}^{(z)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(z-1)} + w_{kj} \} \quad \leftarrow \text{include pure } l_{ij}^{(z-1)} \text{ perché } w_{jj}=0.$$

Oss: Se il grafo $G=(V, E)$ non contiene cicli di peso negativo, allora i cammini minimi (se esistono) sono semplici.

Pertanto, un cammino minimo contiene al più $n-1$ archi.

$$\text{Quindi, } \delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots$$